INFRASTRUCTURE & DEPLOYMENT PLAN

TimeSync on Google Cloud Platform

Target GCP architecture, managed services, and configuration needed to take the TimeSync timesheet & approval platform from local development to a production deployment.

Application: TimeSync — Enterprise Timesheet, Approval & Reporting Platform (Vivid Edge Integration COE)

Current stack: React + TypeScript SPA · Node.js/Express API · Prisma ORM · SQLite (dev)

Target stack: Cloud Run · Cloud SQL for MySQL · Cloud Storage + Cloud CDN · Secret Manager

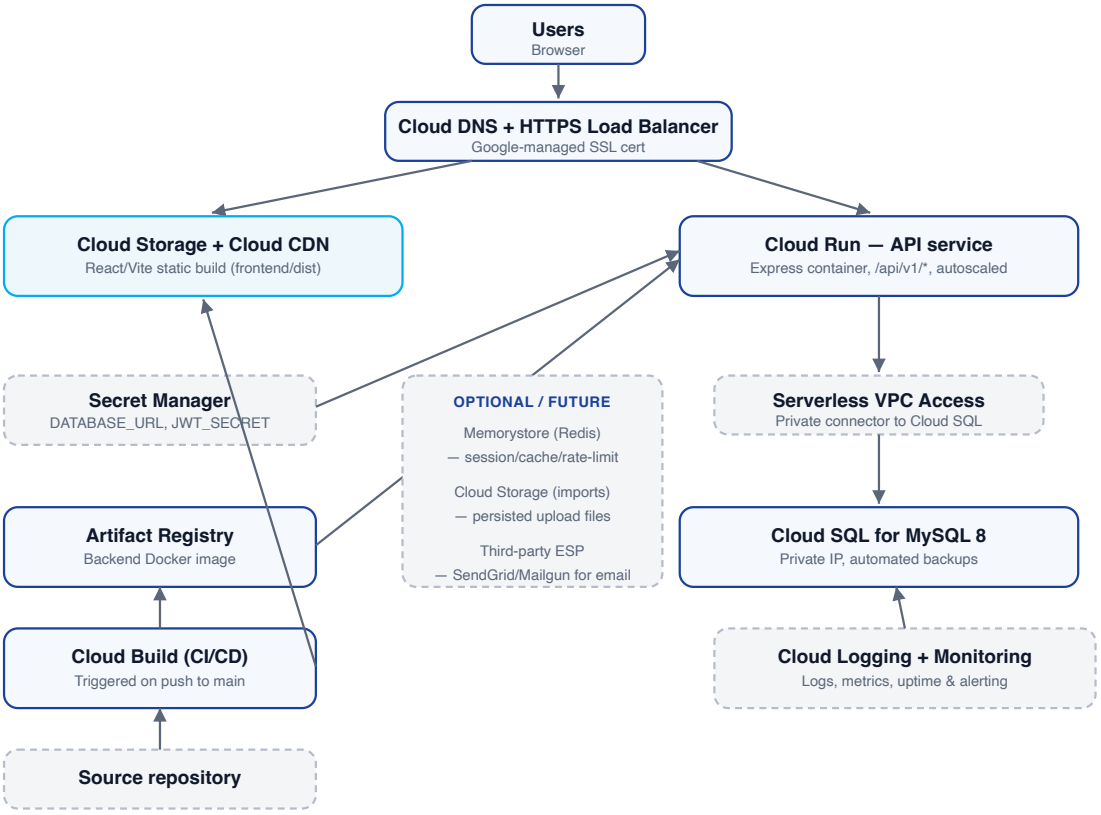
Prepared: September 2026

1 Overview

TimeSync today runs as two local processes — a stateless Express API and a static React SPA — backed by a single SQLite file. Nothing about the codebase is GCP-specific yet; this document maps each local piece to a managed GCP equivalent, lists the services that need to be provisioned, and calls out the gaps (email delivery, secrets, SSO) that exist today and should be closed before a real production launch.

> **Why this maps cleanly to GCP:** the backend is already a single stateless container-shaped process with no local disk state and no in-process session store, and the database schema was written to be MySQL-compatible from day one (see `SCHEMA_REFERENCE.md` §3) — switching the Prisma `provider` from `sqlite` to `mysql` is a two-line change. That combination is exactly what Cloud Run + Cloud SQL is built for.

2 Target architecture



Solid boxes are required services; dashed boxes are supporting/optional pieces (§5–§6).

3 Component mapping

LOCAL COMPONENT TODAY	GCP SERVICE	NOTES
React/Vite SPA (`frontend/` , served by Vite dev server)	Cloud Storage bucket + Cloud CDN (or Firebase Hosting)	Build output (`npm run build` → `frontend/dist/`) is static; no server-side rendering needed
Express API (`backend/` , `tsx watch` / `node dist/index.js`)	Cloud Run	Stateless container, scales to zero, no code changes required beyond a Dockerfile
SQLite (`backend/prisma/dev.db`)	Cloud SQL for MySQL 8	Change `provider` in `schema.prisma` to `mysql` , point `DATABASE_URL` at the instance
`backend/.env` (plaintext file)	Secret Manager	`DATABASE_URL` , `JWT_SECRET` injected as env vars at Cloud Run deploy time
Manual `npm run build`	Cloud Build + Artifact Registry	Builds the backend Docker image and the frontend static bundle on every push
`node-cron` in-process scheduler	Cloud Run (same container) or Cloud Scheduler + Cloud Run job	See §7 — in-process cron doesn't fit Cloud Run's scale-to-zero model as-is
console.log "email" placeholder	Not a GCP service — needs a third-party ESP	See §6 gap list

4 Required GCP services

Compute — Cloud Run (backend API)

- One service, built from a `Dockerfile` in `backend/` (Node 20 base image, `npm run build` , `node dist/index.js`)
- Concurrency: default (80) is fine — the API has no long-lived connections
- Min instances: 0 for cost, or 1 to avoid cold starts on the reminder/approval endpoints
- Memory/CPU: 512 MiB / 1 vCPU is a reasonable starting point for this workload
- Env vars `PORT` , `CORS_ORIGIN` set directly; `DATABASE_URL` and `JWT_SECRET` mounted from Secret Manager

Hosting — Cloud Storage + Cloud CDN (frontend)

- Bucket configured for static website hosting, fronted by Cloud CDN via the same Load Balancer as the API
- Path-based routing on the Load Balancer: `/api/*` → Cloud Run backend, everything else → the bucket

- Alternative: Firebase Hosting, which bundles CDN + managed SSL + a simpler CLI deploy if a separate Load Balancer isn't wanted

Database — Cloud SQL for MySQL

- MySQL 8.x, private IP only (no public IP) — reached from Cloud Run via a Serverless VPC Access connector
- Automated daily backups + point-in-time recovery enabled
- Sizing: `db-custom-1-3840` (1 vCPU / 3.75 GB) comfortably covers this app's load pattern; scale up only if reporting queries grow heavy

Secrets — Secret Manager

- `DATABASE_URL`, `JWT_SECRET` stored as secrets, referenced (not copied) by the Cloud Run service
- Cloud Run's service account granted `roles/secretmanager.secretAccessor` — nothing else

CI/CD — Cloud Build + Artifact Registry

- Build trigger on push to the main branch: build the backend Docker image, push to Artifact Registry, deploy the new revision to Cloud Run; build the frontend and sync `dist/` to the Cloud Storage bucket
- `npx prisma migrate deploy` (see §7) run as a Cloud Build step before the new revision receives traffic

Networking & security

- Serverless VPC Access connector for Cloud Run → Cloud SQL private-IP traffic
- Google-managed SSL certificate on the Load Balancer / Firebase Hosting domain
- A dedicated service account per environment (dev/staging/prod), least-privilege IAM roles only

Observability — Cloud Logging + Cloud Monitoring

- Cloud Run and Cloud SQL logs flow to Cloud Logging automatically — no code changes needed
- Uptime check against `/health` (already implemented) + alerting policy on error rate / latency

5 Environment configuration

Same variables as local dev (`backend/.env`), sourced from Secret Manager instead of a file:

VARIABLE	PURPOSE	PRODUCTION VALUE
`DATABASE_URL`	Prisma datasource	Cloud SQL MySQL connection string (via VPC connector or Cloud SQL Auth Proxy sidecar)
`JWT_SECRET`	Signs/verifies access tokens	Newly generated, high-entropy secret — never reuse the local dev value
`JWT_EXPIRES_IN`	Access token lifetime	8h (unchanged), revisit if a refresh-token flow is added later
`PORT`	API listen port	8080 (Cloud Run's expected default)

VARIABLE	PURPOSE	PRODUCTION VALUE
`CORS_ORIGIN`	Allowed frontend origin	The production frontend domain (not localhost)

6 Gaps to close before production

These exist in the current build as documented, intentional stubs (see `SCHEMA_REFERENCE.md` §10 and `INFRASTRUCTURE.md`) — none of them are GCP configuration, they're application-level work:

> **Email delivery** — reminder/escalation notifications and any "new user" credentials are only ever written as in-app notifications today; "sending" email is a `console.log` placeholder. GCP has no native transactional email service — plan on a third-party ESP (SendGrid, Mailgun, Postmark) called from the backend, with its API key in Secret Manager.

> **SSO** — auth is password + JWT only. If Azure AD/Okta/Google Workspace SSO is required, evaluate **Identity Platform** (GCP's managed IdP integration) or Cloud IAP in front of the Load Balancer.

> **Background scheduler** — the daily reminder sweep runs via `node-cron` inside the same long-lived process. Cloud Run scales to zero and isn't meant to keep a cron timer alive; replace it with **Cloud Scheduler** hitting a dedicated authenticated endpoint (e.g. `POST /internal/reminder-sweep`) once a day, or a separate Cloud Run job.

7 Suggested starting tiers

Pilot / low-traffic

- Cloud Run: min instances 0, 512 MiB / 1 vCPU
- Cloud SQL: `db-f1-micro` shared-core, single zone
- No CDN — serve frontend directly from Cloud Storage

Production

- Cloud Run: min instances 1–2, 1 GiB / 1 vCPU, autoscale on concurrency
- Cloud SQL: `db-custom-1-3840`, regional (HA) with automated backups
- Cloud CDN in front of the frontend bucket, managed SSL, Cloud Armor for basic WAF/rate-limiting

Exact sizing should be revisited once real usage (concurrent users, submission-day traffic spikes, report generation frequency) is known — this is a starting point, not a capacity plan.

8 Related documentation

- `README.md` — local setup, run commands, tech stack
- `INFRASTRUCTURE.md` — current local services, ports, and what's stubbed vs. real
- `SCHEMA_REFERENCE.md` §3 — the SQLite→MySQL migration path this deployment relies on

- `SCHEMA_REFERENCE.md` §10 — full stubbed-vs-real index (email, Redis, S3, SSO)